

hw1_1

September 22, 2026

0.0.1 a_tensor_initialization.py

```
[1]: import torch
```

```
[2]: # torch.Tensor class
t1 = torch.Tensor([1, 2, 3], device='cpu') #! floating dtype
print(t1.dtype)    # >>> torch.float32
print(t1.device)   # >>> cpu
print(t1.requires_grad) # >>> False
print(t1.size())  # torch.Size([3])
print(t1.shape)   # torch.Size([3])
```

```
torch.float32
cpu
False
torch.Size([3])
torch.Size([3])
```

```
[3]: #! gpu device GPU
# t1_cuda = t1.to(torch.device('cuda'))
# t1_cuda = t1.cuda()
#! CPU
t1_cpu = t1.cpu()
```

```
[4]: #! [ ] CUDA GPU CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
t1 = t1.to(device)
print(t1.device) # >>> cuda:0 (GPU ) / cpu (GPU )
```

```
cpu
```

```
[5]: # torch.tensor function
t2 = torch.tensor([1, 2, 3], device='cpu') #!
print(t2.dtype) # >>> torch.int64
print(t2.device) # >>> cpu
print(t2.requires_grad) # >>> False
print(t2.size()) # torch.Size([3])
print(t2.shape) # torch.Size([3])
```

```
torch.int64
cpu
False
torch.Size([3])
torch.Size([3])
```

```
[6]: #! gpu device GPU
      # t2_cuda = t2.to(torch.device('cuda'))
      # t2_cuda = t2.cuda()
      #! cpu
      t2_cpu = t2.cpu()
```

```
[7]: #! 0 ( )
      a1 = torch.tensor(1) # shape: torch.Size([]),
      ↪ ndims(=rank): 0
      print(a1.shape, a1.ndim)
```

```
torch.Size([]) 0
```

```
[8]: #! 1 ( )
      a2 = torch.tensor([1]) # shape: torch.Size([1]),
      ↪ ndims(=rank): 1
      print(a2.shape, a2.ndim)
```

```
torch.Size([1]) 1
```

```
[9]: #! n
      a3 = torch.tensor([1, 2, 3, 4, 5]) # shape: torch.Size([5]), ndims(=rank): 1
      print(a3.shape, a3.ndim)
```

```
torch.Size([5]) 1
```

```
[10]: #! 2 ( )
      a4 = torch.tensor([[1], [2], [3], [4], [5]]) # shape: torch.Size([5, 1]),
      ↪ ndims(=rank): 2
      print(a4.shape, a4.ndim)
```

```
torch.Size([5, 1]) 2
```

```
[11]: a5 = torch.tensor([ # shape: torch.Size([3, 2]), ndims(=rank): 2
      [1, 2],
      [3, 4],
      [5, 6]
    ])
      print(a5.shape, a5.ndim)
```

```
torch.Size([3, 2]) 2
```

```
[12]: a6 = torch.tensor([ # shape: torch.Size([3, 2, 1]),
      ↪ ndims(=rank): 3
```

```

    [[1], [2]],
    [[3], [4]],
    [[5], [6]]
])
print(a6.shape, a6.ndim)

```

torch.Size([3, 2, 1]) 3

```

[13]: a7 = torch.tensor([                                # shape: torch.Size([3, 1, 2, 1]),
    ↪ndims(=rank): 4
    [[[1], [2]]],
    [[[3], [4]]],
    [[[5], [6]]]
])
print(a7.shape, a7.ndim)

```

torch.Size([3, 1, 2, 1]) 4

```

[14]: a8 = torch.tensor([                                # shape: torch.Size([3, 1, 2, 3]),
    ↪ndims(=rank): 4
    [[[1, 2, 3], [2, 3, 4]]],
    [[[3, 1, 1], [4, 4, 5]]],
    [[[5, 6, 2], [6, 3, 1]]]
])
print(a8.shape, a8.ndim)

```

torch.Size([3, 1, 2, 3]) 4

```

[15]: a9 = torch.tensor([                                # shape: torch.Size([3, 1, 2, 3, 1]),
    ↪ndims(=rank): 5
    [[[[1], [2], [3]], [[2], [3], [4]]]],
    [[[[3], [1], [1]], [[4], [4], [5]]]],
    [[[[5], [6], [2]], [[6], [3], [1]]]]
])
print(a9.shape, a9.ndim)

```

torch.Size([3, 1, 2, 3, 1]) 5

```

[16]: a10 = torch.tensor([                               # shape: torch.Size([4, 5]), ndims(=rank):
    ↪2
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
    [1, 2, 3, 4, 5],
])
print(a10.shape, a10.ndim)

```

torch.Size([4, 5]) 2

```
[17]: a10 = torch.tensor([                # shape: torch.Size([4, 1, 5]),
    ↪ndims(=rank): 3
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
    [[1, 2, 3, 4, 5]],
])
print(a10.shape, a10.ndim)
```

torch.Size([4, 1, 5]) 3

```
[18]: #! Tensor
try:
    a11 = torch.tensor([
        [[[1, 2, 3], [4, 5]]],
        [[[1, 2, 3], [4, 5]]],
        [[[1, 2, 3], [4, 5]]],
        [[[1, 2, 3], [4, 5]]]
    ])
except ValueError as error:
    print("      Tensor      ")
    print(type(error).__name__, error)
```

Tensor
ValueError expected sequence of length 3 at dim 3 (got 2)

- torch.tensor() torch.Tensor() tensor
- torch.tensor() dtype torch.Tensor() tensor
- tensor shape

0.0.2 b_tensor_initialization_copy.py

```
[19]: import torch
import numpy as np
```

```
[20]: #! list Tensor
#!
l1 = [1, 2, 3]
t1 = torch.Tensor(l1)

l2 = [1, 2, 3]
t2 = torch.tensor(l2)

l3 = [1, 2, 3]
t3 = torch.as_tensor(l3)
```

```
11[0] = 100
12[0] = 100
13[0] = 100
```

```
print(t1)
print(t2)
print(t3)
```

```
tensor([1., 2., 3.])
tensor([1, 2, 3])
tensor([1, 2, 3])
```

```
[21]: #! NumPy Tensor
14 = np.array([1, 2, 3])
t4 = torch.Tensor(14)

15 = np.array([1, 2, 3])
t5 = torch.tensor(15)

16 = np.array([1, 2, 3])
t6 = torch.as_tensor(16) #! np.ndarray torch.as_tensor()

14[0] = 100
15[0] = 100
16[0] = 100

print(t4)
print(t5)
print(t6)
```

```
tensor([1., 2., 3.])
tensor([1, 2, 3])
tensor([100, 2, 3])
```

- Python list Tensor list
- torch.tensor(), torch.as_tensor()
- NumPy torch.as_tensor() Tensor

0.0.3 c_tensor_initialization_with_size.py

```
[22]: import torch
```

```
[23]: #! torch.ones(): 1 tensor
t1 = torch.ones(size=(5,)) # or torch.ones(5)
print(t1)
```

```

#! torch.ones_like(): input tensor shape, dtype    ones tensor
t1_like = torch.ones_like(input=t1)
print(t1_like)

```

```

tensor([1., 1., 1., 1., 1.])
tensor([1., 1., 1., 1., 1.])

```

```

[24]: #! torch.zeros():      0 tensor
t2 = torch.zeros(size=(6,)) # or torch.zeros(6)
print(t2)

#! torch.zeros_like(): input tensor shape, dtype    zeros tensor
t2_like = torch.zeros_like(input=t2)
print(t2_like)

```

```

tensor([0., 0., 0., 0., 0., 0.])
tensor([0., 0., 0., 0., 0., 0.])

```

```

[25]: #! torch.empty():
#!
#!      0      0
t3 = torch.empty(size=(4,)) # or torch.empty(4)
print(t3)

#! torch.empty_like(): input tensor shape, dtype    empty tensor
t3_like = torch.empty_like(input=t3)
print(t3_like)

```

```

tensor([0., 0., 0., 0.])
tensor([0., 0., 0., 0.])

```

```

[26]: #! torch.eye(n): n*n
t4 = torch.eye(n=3)
print(t4)

```

```

tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])

```

```

[27]: #! [ ]    one-hot encoding
labels = torch.tensor([1, 2, 0])    # (3, )
eye = torch.eye(3)
one_hot_labels = eye[labels]        # (3, 3)

print(one_hot_labels)

```

```

tensor([[0., 1., 0.],
        [0., 0., 1.],
        [1., 0., 0.]])

```

- `ones()`, `zeros()`, `empty()` `shape` `Tensor` , `eye()`
- `_like` `Tensor` `shape`, `dtype`, `device` `Tensor`
- `empty()` 0

0.0.4 `d_tensor_initialization_random_values.py`

```
[28]: import torch
```

```
[29]: #! torch.randint():
t1 = torch.randint(low=10, high=20, size=(1, 2))    #! low    high
print(t1)

tensor([[17, 14]])
```

```
[30]: #! torch.rand():
t2 = torch.rand(size=(1, 3))    #! x~U(0, 1)
print(t2)

tensor([[0.8091, 0.2600, 0.5247]])
```

```
[31]: #! torch.randn():
t3 = torch.randn(size=(1, 3))    #! x~N(0, 1)
print(t3)

tensor([[ 0.4513, -0.2947, 0.2999]])
```

```
[32]: #! torch.normal():    mean,    std
t4 = torch.normal(mean=10.0, std=1.0, size=(3, 2))    #! x~N(10.0, 1.0)
print(t4)

tensor([[ 9.4818, 8.9648],
        [11.9903, 11.2735],
        [ 9.9831, 10.0050]])
```

```
[33]: #! torch.linspace(): start   end   steps   element        (1    )
t5 = torch.linspace(start=0.0, end=5.0, steps=7)
print(t5)

tensor([0.0000, 0.8333, 1.6667, 2.5000, 3.3333, 4.1667, 5.0000])
```

```
[34]: #! torch.arange(): start    end
t6 = torch.arange(5)
print(t6, t6.dtype)

#! [    ]
t6 = torch.arange(5.)
print(t6, t6.dtype)
```

```
tensor([0, 1, 2, 3, 4]) torch.int64
tensor([0., 1., 2., 3., 4.]) torch.float32
```

```
[35]: #!/ torch.manual_seed():
torch.manual_seed(1729)
random1 = torch.rand(2, 3)
random2 = torch.rand(2, 3)
print(random1)
print(random2)
```

```
tensor([[0.3126, 0.3791, 0.3087],
        [0.0736, 0.4216, 0.0691]])
tensor([[0.2332, 0.4047, 0.2162],
        [0.9927, 0.4128, 0.5938]])
```

```
[36]: #!/
torch.manual_seed(1729)
random3 = torch.rand(2, 3)
random4 = torch.rand(2, 3)
print(random3)
print(random4)
```

```
tensor([[0.3126, 0.3791, 0.3087],
        [0.0736, 0.4216, 0.0691]])
tensor([[0.2332, 0.4047, 0.2162],
        [0.9927, 0.4128, 0.5938]])
```

```
[37]: #!/ [ ] mini-batch class label
torch.manual_seed(42)
inputs = torch.randn(32, 784)
labels = torch.randint(0, 10, size=(32,))

print(inputs.shape, inputs.dtype)
print(labels.shape, labels.dtype)
```

```
torch.Size([32, 784]) torch.float32
torch.Size([32]) torch.int64
```

- `randint()` , `rand()` 0 1 , `randn()`
- `manual_seed()`
- `size` Tensor shape

0.0.5 e_tensor_type_conversion.py

```
[38]: import torch
```

```
[39]: a = torch.ones((2, 3))  #! dtype torch.float32
      b = torch.ones((2, 3), dtype=torch.int16)
      print(a)
      print(b)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1, 1, 1],
        [1, 1, 1]], dtype=torch.int16)
```

```
[40]: c = torch.rand((2, 3), dtype=torch.float64) * 20.  #! 20.0 0 20
      print(c)
```

```
tensor([[11.5115,  7.0453, 18.3587],
        [13.8709, 19.5962, 12.1074]], dtype=torch.float64)
```

```
[41]: d = b.to(torch.int32)  #! torch.tensor().to() device dtype
      print(d)
```

```
tensor([[1, 1, 1],
        [1, 1, 1]], dtype=torch.int32)
```

```
[42]: double_d = torch.ones(10, 2, dtype=torch.double)  #! torch.double torch.
      ↪ float64
      short_e = torch.tensor([[1, 2]], dtype=torch.short) #! torch.short torch.int16 ↪

      print(double_d.dtype)
      print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
[43]: #! dtype 1: torch.tensor().double(), torch.tensor().short()
      double_d = torch.zeros(10, 2).double()
      short_e = torch.ones(10, 2).short()

      print(double_d.dtype)
      print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
[44]: #! dtype 2: torch.tensor().to()
      double_d = torch.zeros(10, 2).to(torch.double)
      short_e = torch.ones(10, 2).to(dtype=torch.short)

      print(double_d.dtype)
      print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
[45]: #! dtype      3: torch.tensor().type()
double_d = torch.zeros(10, 2).type(torch.double)
short_e = torch.ones(10, 2).type(dtype=torch.short)

print(double_d.dtype)
print(short_e.dtype)
```

```
torch.float64
torch.int16
```

```
[46]: double_f = torch.rand(5, dtype=torch.double)      # torch.float64
short_g = double_f.to(torch.short)                    # torch.int16
print((double_f * short_g).dtype)                     #! dtype PyTorch
```

```
torch.float64
```

- `.double()`, `.short()`, `.to()`, `.type()` dtype dtype shape
- Tensor dtype
- dtype PyTorch dtype

0.0.6 f_tensor_operations.py

```
[47]: import torch
```

```
[48]: #! shape dtype Tensor element-wise
t1 = torch.ones(size=(2, 3))
t2 = torch.ones(size=(2, 3))

print(t1)
print(t2)
```

```
tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

```
[49]: #! t1 t2 shape element-wise
#! element-wise
t3 = torch.add(t1, t2)
t4 = t1 + t2 #! torch.add()
print(t3)
print(t4)
```

```

tensor([[2., 2., 2.],
        [2., 2., 2.]])
tensor([[2., 2., 2.],
        [2., 2., 2.]])

```

```

[50]: #! element-wise

t5 = torch.sub(t1, t2)
t6 = t1 - t2 #! torch.sub()
print(t5)
print(t6)

```

```

tensor([[0., 0., 0.],
        [0., 0., 0.]])
tensor([[0., 0., 0.],
        [0., 0., 0.]])

```

```

[51]: #! element-wise

t7 = torch.mul(t1, t2)
t8 = t1 * t2 #! torch.mul()
print(t7)
print(t8)

```

```

tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])

```

```

[52]: #! element-wise

t9 = torch.div(t1, t2)
t10 = t1 / t2 #! torch.div()
print(t9)
print(t10)

```

```

tensor([[1., 1., 1.],
        [1., 1., 1.]])
tensor([[1., 1., 1.],
        [1., 1., 1.]])

```

- `add()`, `sub()`, `mul()`, `div()` element-wise
- `+`, `-`, `*`, `/`
- `shape` `shape` `shape` broadcasting

0.0.7 g_tensor_operations_mm.py

```
[53]: import torch
```

```
[54]: # torch.dot():  
# (m,) * (m,) → ()  
t1 = torch.dot(  
    torch.tensor([2, 3]), torch.tensor([2, 1])  
)  
print(t1, t1.size())
```

```
tensor(7) torch.Size([])
```

```
[55]: t2 = torch.randn(2, 3)  
t3 = torch.randn(3, 2)  
  
# torch.mm():  
# (m, n) * (n, p) → (m, p)  
t4 = torch.mm(t2, t3)  
print(t2, t2.size())  
print(t3, t3.size())  
print(t4, t4.size())
```

```
tensor([[ 0.1249, -1.5781, -0.6635],  
        [ 0.9366, -0.5759,  1.0020]]) torch.Size([2, 3])  
tensor([[ -0.1700,  0.8390],  
        [ 0.8995,  0.0264],  
        [ 1.0481,  0.5525]]) torch.Size([3, 2])  
tensor([[ -2.1361, -0.3034],  
        [ 0.3730,  1.3243]]) torch.Size([2, 2])
```

```
[56]: t5 = torch.randn(10, 3, 4)  
t6 = torch.randn(10, 4, 5)  
  
# torch.bmm():  
# (B, m, n) * (B, n, p) → (B, m, p)  
t7 = torch.bmm(t5, t6)  
print(t7.size())
```

```
torch.Size([10, 3, 5])
```

- dot() 1 Tensor 0 Tensor
- mm() 2 (m, n) × (n, p) shape (m, p)
- bmm() 3 Tensor batch batch

0.0.8 h_tensor_operations_matmul.py

```
[57]: import torch
```

```
[58]: #! torch.matmul():
#      ×
t1 = torch.randn(3) # (m,)
t2 = torch.randn(3) # (m,)
print(torch.matmul(t1, t2).size()) # (m,) * (m,) → ()

torch.Size([])
```

```
[59]: #      ×
t3 = torch.randn(3, 4) # (m, n)
t4 = torch.randn(4)    # (n,)
print(torch.matmul(t3, t4)) # (m, n) * (n,) → (m,)

tensor([-2.2168,  0.8540,  0.9579])
```

```
[60]: #      ×
t5 = torch.randn(10, 3, 4) # (B, m, n)
t6 = torch.randn(4)        # (n,)
print(torch.matmul(t5, t6).size()) # (B, m, n) * (n,) → (B, m)

torch.Size([10, 3])
```

```
[61]: #      ×      : bmm
t7 = torch.randn(10, 3, 4) # (B, m, n)
t8 = torch.randn(10, 4, 5) # (B, n, p)
print(torch.matmul(t7, t8).size()) # (B, m, n) * (B, n, p) → (B, m, p)

torch.Size([10, 3, 5])
```

```
[62]: #      ×      : broadcasting
t9 = torch.randn(10, 3, 4) # (B, m, n)
t10 = torch.randn(4, 5)    # (n, p)
print(torch.matmul(t9, t10).size()) # (B, m, n) * (n, p) → (B, m, p)

torch.Size([10, 3, 5])
```

```
[63]: #! [ ]
features = torch.randn(32, 128)
weights = torch.randn(128, 10)
bias = torch.randn(10)

logits = torch.matmul(features, weights) + bias
print(logits.shape) # >>> torch.Size([32, 10])

torch.Size([32, 10])
```

```
[64]: #! [ ] attention score
queries = torch.randn(4, 8, 16) # (batch, sequence, hidden)
keys = torch.randn(4, 8, 16) # (batch, sequence, hidden)

attention_scores = torch.bmm(queries, keys.transpose(1, 2))
print(attention_scores.shape) # >>> torch.Size([4, 8, 8])

torch.Size([4, 8, 8])
```

- `matmul()` , , shape
- 2 batch
- bias batch Tensor broadcasting

0.0.9 i_tensor_broadcasting.py

```
[65]: import torch
```

```
[66]: t1 = torch.tensor([1.0, 2.0, 3.0]) #! (3, )
t2 = 2.0 #!
print(t1 * t2) #! broadcasting
```

```
tensor([2., 4., 6.])
```

```
[67]: t3 = torch.tensor([[0, 1], [2, 4], [10, 10]]) #! (3, 2)
t4 = torch.tensor([4, 5]) #! (2,)
print(t3 - t4) #! t4 dim=0 broadcasting
```

```
tensor([[ -4, -4],
        [-2, -1],
        [ 6,  5]])
```

```
[68]: t5 = torch.tensor([[1., 2.], [3., 4.]])
print(t5 + 2.0) # t5.add(2.0)
print(t5 - 2.0) # t5.sub(2.0)
print(t5 * 2.0) # t5.mul(2.0)
print(t5 / 2.0) # t5.div(2.0)
```

```
tensor([[3., 4.],
        [5., 6.]])
tensor([[ -1.,  0.],
        [ 1.,  2.]])
tensor([[2., 4.],
        [6., 8.]])
tensor([[0.5000, 1.0000],
        [1.5000, 2.0000]])
```

```
[69]: def normalize(x):
        return x / 255  #! broadcasting
```

```
t6 = torch.randn(3, 28, 28)
print(normalize(t6).size())
```

```
torch.Size([3, 28, 28])
```

```
[70]: t7 = torch.tensor([[1, 2], [0, 3]]) # (2, 2)
        t8 = torch.tensor([[3, 1]])      # (1, 2)
        t9 = torch.tensor([[5], [2]])    # (2, 1)
        t10 = torch.tensor([7])          # (1,)
        print(t7 + t8)  # >>> tensor([[4, 3], [3, 4]])
        print(t7 + t9)  # >>> tensor([[6, 7], [2, 5]])
        print(t8 + t9)  # >>> tensor([[8, 6], [5, 3]])
        print(t7 + t10) # >>> tensor([[ 8, 9], [ 7, 10]])
```

```
tensor([[4, 3],
        [3, 4]])
tensor([[6, 7],
        [2, 5]])
tensor([[8, 6],
        [5, 3]])
tensor([[ 8,  9],
        [ 7, 10]])
```

```
[71]: t11 = torch.ones(4, 3, 2)
        t12 = t11 * torch.rand(3, 2)  #! (1, 3, 2)    dim=1,2    -> dim=0,2
        ↪broadcasting
        print(t12.shape)
```

```
torch.Size([4, 3, 2])
```

```
[72]: t13 = torch.ones(4, 3, 2)
        t14 = t13 * torch.rand(3, 1)  #! (1, 3, 1)    dim=1    -> dim=0,2
        ↪broadcasting
        print(t14.shape)
```

```
torch.Size([4, 3, 2])
```

```
[73]: t15 = torch.ones(4, 3, 2)
        t16 = t15 * torch.rand(1, 2)  #! (1, 1, 2)    dim=2    -> dim=0,1
        ↪broadcasting
        print(t16.shape)
```

```
torch.Size([4, 3, 2])
```

```
[74]: t17 = torch.ones(5, 3, 4, 1)
        t18 = torch.rand(3, 1, 1)  #! (1, 3, 1, 1)    dim=1,3    -> dim=0,2
        ↪broadcasting
```

```
print((t17 + t18).size())
```

```
torch.Size([5, 3, 4, 1])
```

```
[75]: t19 = torch.empty(5, 1, 4, 1)    #! dim=1    1 -> dim=1 broadcasting  
      t20 = torch.empty(3, 1, 1)      #! (1, 3, 1, 1)    dim=3    -> dim=0,2  
      ↪broadcasting  
      print((t19 + t20).size())
```

```
torch.Size([5, 3, 4, 1])
```

```
[76]: t21 = torch.empty(1)             #! (1, 1, 1)    dim=1    -> dim=0,2 broadcasting  
      t22 = torch.empty(3, 1, 7)  
      print((t21 + t22).size())
```

```
torch.Size([3, 1, 7])
```

```
[77]: t23 = torch.ones(3, 3, 3)  
      t24 = torch.ones(3, 1, 3)    #! dim=0,2    -> dim=1 broadcasting  
      print((t23 + t24).size())
```

```
torch.Size([3, 3, 3])
```

```
[78]: #! Broadcasting :    shape  
      try:  
          t25 = torch.empty(5, 2, 4, 1)  
          t26 = torch.empty(3, 1, 1) #! (1, 3, 1, 1)    dim=1    -> RuntimeError  
          print((t25 + t26).size())  
      except RuntimeError as error:  
          print("Broadcasting :    shape    ")  
          print(type(error).__name__, error)
```

```
Broadcasting :    shape
```

```
RuntimeError The size of tensor a (2) must match the size of tensor b (3) at  
non-singleton dimension 1
```

```
[79]: #!    5  
      t27 = torch.ones(4) * 5  
      print(t27)
```

```
tensor([5., 5., 5., 5.])
```

```
[80]: #!  
      t28 = torch.pow(t27, 2)  
      print(t28)
```

```
tensor([25., 25., 25., 25.])
```

```
[81]: exp = torch.arange(1., 5.)  
      a = torch.arange(1., 5.)  
      t29 = torch.pow(a, exp)
```

```
print(t29) # >>> tensor([ 1.,  4., 27., 256.])
```

```
tensor([ 1.,  4., 27., 256.])
```

```
[82]: #! [ ] mean std broadcasting
```

```
images = torch.rand(32, 3, 224, 224)
```

```
mean = torch.tensor([0.485, 0.456, 0.406]).reshape(1, 3, 1, 1)
```

```
std = torch.tensor([0.229, 0.224, 0.225]).reshape(1, 3, 1, 1)
```

```
normalized_images = (images - mean) / std
```

```
print(normalized_images.size()) # >>> torch.Size([32, 3, 224, 224])
```

```
torch.Size([32, 3, 224, 224])
```

- Broadcasting shape 1
- shape 1
- Broadcasting

0.0.10 j_tensor_indexing_slicing.py

```
[83]: import torch
```

```
[84]: x = torch.tensor(  
    [[0, 1, 2, 3, 4],  
     [5, 6, 7, 8, 9],  
     [10, 11, 12, 13, 14]]  
)  
  
print(x[1]) # >>> tensor([5, 6, 7, 8, 9])  
print(x[:, 1]) # >>> tensor([1, 6, 11])  
print(x[1, 2]) # >>> tensor(7)  
print(x[:, -1]) # >>> tensor([4, 9, 14])
```

```
tensor([5, 6, 7, 8, 9])
```

```
tensor([ 1,  6, 11])
```

```
tensor(7)
```

```
tensor([ 4,  9, 14])
```

```
[85]: print(x[1:]) # >>> tensor([[ 5,  6,  7,  8,  9], [10, 11, 12, 13, 14]])  
print(x[1:, 3:]) # >>> tensor([[ 8,  9], [13, 14]])
```

```
tensor([[ 5,  6,  7,  8,  9],  
        [10, 11, 12, 13, 14]])
```

```
tensor([[ 8,  9],  
        [13, 14]])
```

```
[86]: #! slicing      1~3      2      1      y
y = torch.zeros((6, 6))
y[1:4, 2] = 1
print(y)
print(y[1:4, 1:4])
```

```
tensor([[0., 0., 0., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 1., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0., 0.]])
tensor([[0., 1., 0.],
        [0., 1., 0.],
        [0., 1., 0.]])
```

```
[87]: #!      slicing ,      Tensor
z = torch.tensor(
    [[1, 2, 3, 4],
     [2, 3, 4, 5],
     [5, 6, 7, 8]]
)
print(z[:2])
print(z[1:, 1:3])
print(z[:, 1:])
```

```
tensor([[1, 2, 3, 4],
        [2, 3, 4, 5]])
tensor([[3, 4],
        [6, 7]])
tensor([[2, 3, 4],
        [3, 4, 5],
        [6, 7, 8]])
```

```
[88]: #! 1      , 1      3      0      z
z[1:, 1:3] = 0
print(z)
```

```
tensor([[1, 2, 3, 4],
        [2, 0, 0, 5],
        [5, 0, 0, 8]])
```

```
[89]: #! [ ] batch class  logit
logits = torch.randn(32, 10)
first_sample = logits[0]
first_five_samples = logits[:5]
class_zero_scores = logits[:, 0]

print(first_sample.shape)      # >>> torch.Size([10])
```

```
print(first_five_samples.shape) # >>> torch.Size([5, 10])
print(class_zero_scores.shape) # >>> torch.Size([32])
```

```
torch.Size([10])
torch.Size([5, 10])
torch.Size([32])
```

- slicing
- :
- slicing Tensor

0.0.11 k_tensor_reshaping.py

```
[90]: import torch
```

```
[91]: t1 = torch.tensor([[1, 2, 3], [4, 5, 6]]) # (2, 3)
t2 = t1.view(3, 2) # (2, 3) -> (3, 2)
t3 = t1.reshape(1, 6) # (2, 3) -> (1, 6)
t2[0, 0] = 100
t3[0, 1] = 200
print(t1)
print(t2)
print(t3)
```

```
tensor([[100, 200, 3],
        [ 4, 5, 6]])
tensor([[100, 200],
        [ 3, 4],
        [ 5, 6]])
tensor([[100, 200, 3, 4, 5, 6]])
```

```
[92]: t4 = torch.arange(8).view(2, 4) # Shape becomes (2, 4)
t5 = torch.arange(6).view(2, 3) # Shape becomes (2, 3)
print(t4)
print(t5)
```

```
tensor([[0, 1, 2, 3],
        [4, 5, 6, 7]])
tensor([[0, 1, 2],
        [3, 4, 5]])
```

```
[93]: # Original tensor with shape (1, 3, 1)
t6 = torch.tensor([[[1], [2], [3]])]

# 1
t7 = t6.squeeze() # Shape becomes (3,)
```

```
# dim 0
t8 = t6.squeeze(0) # Shape becomes (3, 1)
print(t7)
print(t8)
```

```
tensor([1, 2, 3])
tensor([[1],
        [2],
        [3]])
```

```
[94]: # Original tensor with shape (3,)
t9 = torch.tensor([1, 2, 3])

# dim 1
t10 = t9.unsqueeze(1) # Shape becomes (3, 1)
print(t10)
```

```
tensor([[1],
        [2],
        [3]])
```

```
[95]: t11 = torch.tensor(
    [[1, 2, 3],
     [4, 5, 6]]
) # (2, 3)

# dim 1
t12 = t11.unsqueeze(1) # Shape becomes (2, 1, 3)
print(t12, t12.shape)
```

```
tensor([[[1, 2, 3]],
        [[4, 5, 6]]]) torch.Size([2, 1, 3])
```

```
[96]: # Original tensor with shape (2, 3)
t13 = torch.tensor([1, 2, 3], [4, 5, 6])

# Flatten the tensor
t14 = t13.flatten() # Shape becomes (6,)

print(t14)
```

```
tensor([1, 2, 3, 4, 5, 6])
```

```
[97]: # Original tensor with shape (2, 2, 2)
t15 = torch.tensor([[[1, 2],
                     [3, 4]],
                    [[5, 6],
                     [7, 8]]])
```

```

#! dim 0 flatten
t16 = torch.flatten(t15)    # (8, )

#! dim 1 flatten
t17 = torch.flatten(t15, start_dim=1)    # (2, 4)

print(t16)
print(t17)

```

```

tensor([1, 2, 3, 4, 5, 6, 7, 8])
tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])

```

```

[98]: t18 = torch.randn(2, 3, 5)
      print(t18.shape)

      #! dim 0, 1, 2
      print(torch.permute(t18, (2, 0, 1)).shape)    # (2, 3, 5) -> (5, 2, 3)

      torch.Size([2, 3, 5])
      torch.Size([5, 2, 3])

```

```

[99]: # Original tensor with shape (2, 3)
      t19 = torch.tensor([[1, 2, 3], [4, 5, 6]])

      # Permute the dimensions
      t20 = torch.permute(t19, dims=(0, 1))    # Shape becomes (2, 3) still
      t21 = torch.permute(t19, dims=(1, 0))    # Shape becomes (3, 2)
      print(t20)
      print(t21)

```

```

tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1, 4],
        [2, 5],
        [3, 6]])

```

```

[100]: #!
        t22 = torch.transpose(t19, 0, 1)    # Shape becomes (3, 2)

        print(t22)

```

```

tensor([[1, 4],
        [2, 5],
        [3, 6]])

```

```

[101]: #! 2 (transpose)
        t23 = torch.t(t19)    # Shape becomes (3, 2)

```

```
print(t23)
```

```
tensor([[1, 4],  
        [2, 5],  
        [3, 6]])
```

```
[102]: #! [ ] CNN feature map batch flatten  
feature_maps = torch.randn(32, 64, 7, 7)  
flattened = feature_maps.flatten(start_dim=1)  
  
print(flattened.shape) # >>> torch.Size([32, 3136])
```

```
torch.Size([32, 3136])
```

```
[103]: #! [ ] batch  
single_image = torch.randn(3, 224, 224)  
batched_image = single_image.unsqueeze(0)  
  
print(batched_image.shape) # >>> torch.Size([1, 3, 224, 224])
```

```
torch.Size([1, 3, 224, 224])
```

- view() reshape() shape , view()
- squeeze() 1 unsqueeze() 1
- flatten() , permute() transpose()

0.0.12 l_tensor_concat.py

```
[104]: import torch
```

```
[105]: t1 = torch.zeros([2, 1, 3])  
t2 = torch.zeros([2, 3, 3])  
t3 = torch.zeros([2, 2, 3])  
  
#! dim 1 concatenate  
t4 = torch.cat([t1, t2, t3], dim=1)  
print(t4.shape)
```

```
torch.Size([2, 6, 3])
```

```
[106]: t5 = torch.arange(0, 3) # tensor([0, 1, 2])  
t6 = torch.arange(3, 8) # tensor([3, 4, 5, 6, 7])  
  
#! dim 0 concatenate  
t7 = torch.cat((t5, t6), dim=0)  
print(t7.shape) # >>> torch.Size([8])  
print(t7) # >>> tensor([0, 1, 2, 3, 4, 5, 6, 7])
```

```
torch.Size([8])
tensor([0, 1, 2, 3, 4, 5, 6, 7])
```

```
[107]: t8 = torch.arange(0, 6).reshape(2, 3) # torch.Size([2, 3])
       t9 = torch.arange(6, 12).reshape(2, 3) # torch.Size([2, 3])
```

```
#!/
t10 = torch.cat((t8, t9), dim=0)
print(t10.size()) # >>> torch.Size([4, 3])
print(t10)
# >>> tensor([[ 0,  1,  2],
#              [ 3,  4,  5],
#              [ 6,  7,  8],
#              [ 9, 10, 11]])
```

```
torch.Size([4, 3])
tensor([[ 0,  1,  2],
        [ 3,  4,  5],
        [ 6,  7,  8],
        [ 9, 10, 11]])
```

```
[108]: #!
       t11 = torch.cat((t8, t9), dim=1)
       print(t11.size()) # >>> torch.Size([2, 6])
       print(t11)
       # >>> tensor([[ 0,  1,  2,  6,  7,  8],
#                    [ 3,  4,  5,  9, 10, 11]])
```

```
torch.Size([2, 6])
tensor([[ 0,  1,  2,  6,  7,  8],
        [ 3,  4,  5,  9, 10, 11]])
```

```
[109]: t12 = torch.arange(0, 6).reshape(2, 3) # torch.Size([2, 3])
       t13 = torch.arange(6, 12).reshape(2, 3) # torch.Size([2, 3])
       t14 = torch.arange(12, 18).reshape(2, 3) # torch.Size([2, 3])
```

```
#!/ 3 dim 0 concatenate
t15 = torch.cat((t12, t13, t14), dim=0)
print(t15.size()) # >>> torch.Size([6, 3])
print(t15)
# >>> tensor([[ 0,  1,  2],
#              [ 3,  4,  5],
#              [ 6,  7,  8],
#              [ 9, 10, 11],
#              [12, 13, 14],
#              [15, 16, 17]])
```

```
torch.Size([6, 3])
tensor([[ 0,  1,  2],
```

```

[ 3,  4,  5],
[ 6,  7,  8],
[ 9, 10, 11],
[12, 13, 14],
[15, 16, 17]])

```

```

[110]: #! 3 dim 1 concatenate
t16 = torch.cat((t12, t13, t14), dim=1)
print(t16.size()) # >>> torch.Size([2, 9])
print(t16)
# >>> tensor([[ 0,  1,  2,  6,  7,  8, 12, 13, 14],
#             [ 3,  4,  5,  9, 10, 11, 15, 16, 17]])

```

```

torch.Size([2, 9])
tensor([[ 0,  1,  2,  6,  7,  8, 12, 13, 14],
        [ 3,  4,  5,  9, 10, 11, 15, 16, 17]])

```

```

[111]: t17 = torch.arange(0, 6).reshape(1, 2, 3) # torch.Size([1, 2, 3])
t18 = torch.arange(6, 12).reshape(1, 2, 3) # torch.Size([1, 2, 3])

```

```

#! 3 2 dim 0 concatenate
t19 = torch.cat((t17, t18), dim=0)
print(t19.size()) # >>> torch.Size([2, 2, 3])
print(t19)
# >>> tensor([[[ 0,  1,  2],
#               [ 3,  4,  5]],
#             [[ 6,  7,  8],
#               [ 9, 10, 11]]])

```

```

torch.Size([2, 2, 3])
tensor([[[ 0,  1,  2],
         [ 3,  4,  5]],
        [[ 6,  7,  8],
         [ 9, 10, 11]]])

```

```

[112]: #! 3 2 dim 1 concatenate
t20 = torch.cat((t17, t18), dim=1)
print(t20.size()) # >>> torch.Size([1, 4, 3])
print(t20)
# >>> tensor([[[ 0,  1,  2],
#               [ 3,  4,  5],
#               [ 6,  7,  8],
#               [ 9, 10, 11]]])

```

```

torch.Size([1, 4, 3])
tensor([[[ 0,  1,  2],
         [ 3,  4,  5],
         [ 6,  7,  8],

```

```
[ 9, 10, 11]])
```

```
[113]: #! 3      2 dim 2      concatenate  
  
t21 = torch.cat((t17, t18), dim=2)  
print(t21.size()) # >>> torch.Size([1, 2, 6])  
print(t21)  
# >>> tensor([[[ 0,  1,  2,  6,  7,  8],  
                [ 3,  4,  5,  9, 10, 11]])]
```

```
torch.Size([1, 2, 6])  
tensor([[[ 0,  1,  2,  6,  7,  8],  
         [ 3,  4,  5,  9, 10, 11]])]
```

```
[114]: #! [  ] feature feature  
image_features = torch.randn(32, 128)  
text_features = torch.randn(32, 256)  
combined_features = torch.cat([image_features, text_features], dim=1)  
  
print(combined_features.shape) # >>> torch.Size([32, 384])
```

```
torch.Size([32, 384])
```

- cat() Tensor
- ,
- (32, 128) (32, 256) dim=1 batch 32 feature 384

0.0.13 m_tensor_stacking.py

```
[115]: import torch
```

```
[116]: t1 = torch.tensor([[1, 2, 3], [4, 5, 6]]) #! (2, 3)  
t2 = torch.tensor([[7, 8, 9], [10, 11, 12]]) #! (2, 3)  
  
#! torch.stack():  
t3 = torch.stack([t1, t2], dim=0) #! (2, 3)␣  
↪-> (2, 2, 3)  
t4 = torch.cat([t1.unsqueeze(dim=0), t2.unsqueeze(dim=0)], dim=0) #! (2, 3)␣  
↪-> (2, 2, 3)  
print(t3, t3.shape, t3.equal(t4))
```

```
tensor([[[ 1,  2,  3],  
         [ 4,  5,  6]],  
        [[ 7,  8,  9],  
         [10, 11, 12]]]) torch.Size([2, 2, 3]) True
```

```
[117]: #! dim 1      stack
t5 = torch.stack([t1, t2], dim=1) #! (2, 3)
      ↪-> (2, 2, 3)
t6 = torch.cat([t1.unsqueeze(dim=1), t2.unsqueeze(dim=1)], dim=1) #! (2, 3)
      ↪-> (2, 2, 3)
print(t5, t5.shape, t5.equal(t6))
```

```
tensor([[[ 1,  2,  3],
          [ 7,  8,  9]],

        [[ 4,  5,  6],
          [10, 11, 12]]]) torch.Size([2, 2, 3]) True
```

```
[118]: #! dim 2      stack
t7 = torch.stack([t1, t2], dim=2)
t8 = torch.cat([t1.unsqueeze(dim=2), t2.unsqueeze(dim=2)], dim=2)
print(t7, t7.shape, t7.equal(t8))
```

```
tensor([[[ 1,  7],
          [ 2,  8],
          [ 3,  9]],

        [[ 4, 10],
          [ 5, 11],
          [ 6, 12]]]) torch.Size([2, 3, 2]) True
```

```
[119]: t9 = torch.arange(0, 3) # tensor([0, 1, 2])
t10 = torch.arange(3, 6) # tensor([3, 4, 5])

print(t9.size(), t10.size())
# >>> torch.Size([3]) torch.Size([3])
```

```
torch.Size([3]) torch.Size([3])
```

```
[120]: t11 = torch.stack((t9, t10), dim=0)
print(t11.size()) # >>> torch.Size([2,3])
print(t11)
# >>> tensor([[0, 1, 2],
              [3, 4, 5]])

t12 = torch.cat((t9.unsqueeze(0), t10.unsqueeze(0)), dim=0)
print(t11.equal(t12))
# >>> True
```

```
torch.Size([2, 3])
tensor([[0, 1, 2],
        [3, 4, 5]])
```

```
True
```

```
[121]: t13 = torch.stack((t9, t10), dim=1)
print(t13.size()) # >>> torch.Size([3,2])
print(t13)
# >>> tensor([[0, 3],
#             [1, 4],
#             [2, 5]])
t14 = torch.cat((t9.unsqueeze(1), t10.unsqueeze(1)), dim=1)
print(t13.equal(t14))
# >>> True
```

```
torch.Size([3, 2])
tensor([[0, 3],
        [1, 4],
        [2, 5]])
```

True

```
[122]: #! [ ] shape stack RuntimeError
try:
    t15 = torch.arange(0, 6).reshape(2, 3)
    t16 = torch.arange(6, 14).reshape(2, 4)
    t17 = torch.stack((t15, t16), dim=0) #! RuntimeError
    print(t17.size())
except RuntimeError as error:
    print("    shape    stack    RuntimeError ")
    print(type(error).__name__, error)
```

```
    shape    stack    RuntimeError
RuntimeError stack expects each tensor to be equal size, but got [2, 3] at entry
0 and [2, 4] at entry 1
```

```
[123]: #! [ ] sample batch
sample1 = torch.randn(3, 224, 224)
sample2 = torch.randn(3, 224, 224)
batch = torch.stack([sample1, sample2], dim=0)

print(batch.shape) # >>> torch.Size([2, 3, 224, 224])
```

```
torch.Size([2, 3, 224, 224])
```

- `stack()` Tensor
- Tensor shape broadcasting
- `stack(..., dim=d)` `unsqueeze(d)` `cat(..., dim=d)`

0.0.14 n_tensor_vstack_hstack.py

```
[124]: import torch
```

```
[125]: t1 = torch.tensor([1, 2, 3])    # (3, )
        t2 = torch.tensor([4, 5, 6])    # (3, )

        #! torch.vstack(): 1      unsqueeze(0)  dim=0      concat, 2      dim=0      concat
        t3 = torch.vstack((t1, t2))      # (2, 3)
        print(t3)
        print(torch.cat((t1.unsqueeze(0), t2.unsqueeze(0)), dim=0).equal(t3))

        tensor([[1, 2, 3],
                [4, 5, 6]])
        True
```

```
[126]: t4 = torch.tensor([[1], [2], [3]]) # (3, 1)
        t5 = torch.tensor([[4], [5], [6]]) # (3, 1)
        t6 = torch.vstack((t4, t5))        # (6, 1)
        print(t6)
        print(torch.cat((t4, t5), dim=0).equal(t6))

        tensor([[1],
                [2],
                [3],
                [4],
                [5],
                [6]])
        True
```

```
[127]: t7 = torch.tensor([
        [[1, 2, 3], [4, 5, 6]],
        [[7, 8, 9], [10, 11, 12]]
    ])
        t8 = torch.tensor([
        [[13, 14, 15], [16, 17, 18]],
        [[19, 20, 21], [22, 23, 24]]
    ])
        t9 = torch.vstack([t7, t8])

        print(t7, t7.shape) # (2, 2, 3)
        print(t8, t8.shape) # (2, 2, 3)
        print(t9, t9.shape) # (4, 2, 3)

        tensor([[[ 1,  2,  3],
                [ 4,  5,  6]],

                [[ 7,  8,  9],
                [10, 11, 12]]]) torch.Size([2, 2, 3])
        tensor([[[13, 14, 15],
                [16, 17, 18]],

                [[19, 20, 21],
```

```

        [22, 23, 24]]]) torch.Size([2, 2, 3])
tensor([[[ 1,  2,  3],
         [ 4,  5,  6]],

        [[ 7,  8,  9],
         [10, 11, 12]],

        [[13, 14, 15],
         [16, 17, 18]],

        [[19, 20, 21],
         [22, 23, 24]]]) torch.Size([4, 2, 3])

```

```

[128]: t10 = torch.tensor([1, 2, 3])    #! (3, )
       t11 = torch.tensor([4, 5, 6])    #! (3, )

       #! torch.hstack(): 1    dim=0, 2    dim=1    concatenate
       t12 = torch.hstack((t10, t11))    #! (6, )
       print(t12, t12.shape)

       print(torch.cat((t10, t11), dim=0).equal(t12))

```

```

tensor([1, 2, 3, 4, 5, 6]) torch.Size([6])
True

```

```

[129]: t13 = torch.tensor([[1], [2], [3]]) #! (3, 1)
       t14 = torch.tensor([[4], [5], [6]]) #! (3, 1)
       t15 = torch.hstack((t13, t14))      #! (3, 2)
       print(t15, t15.shape)

       print(torch.cat((t13, t14), dim=1).equal(t15))

```

```

tensor([[1, 4],
        [2, 5],
        [3, 6]]) torch.Size([3, 2])
True

```

```

[130]: t16 = torch.tensor([
        [[1, 2, 3], [4, 5, 6]],
        [[7, 8, 9], [10, 11, 12]]
    ])
       t17 = torch.tensor([
        [[13, 14, 15], [16, 17, 18]],
        [[19, 20, 21], [22, 23, 24]]
    ])
       print(t16.shape) # (2, 2, 3)
       print(t17.shape) # (2, 2, 3)

```

```
t18 = torch.hstack([t16, t17])
print(t18.shape) # (2, 4, 3)
print(t18)
```

```
torch.Size([2, 2, 3])
torch.Size([2, 2, 3])
torch.Size([2, 4, 3])
tensor([[[ 1,  2,  3],
          [ 4,  5,  6],
          [13, 14, 15],
          [16, 17, 18]],
        [[ 7,  8,  9],
          [10, 11, 12],
          [19, 20, 21],
          [22, 23, 24]]])
```

- `vstack()` 1 Tensor, 2 dim=0
- `hstack()` 1 dim=0, 2 dim=1
- broadcasting

0.0.15

```
2, Tensor 3 . broadcasting shape 1
. cat() stack() shape . view(), reshape(),
permute() . Tensor shape dtype .
```